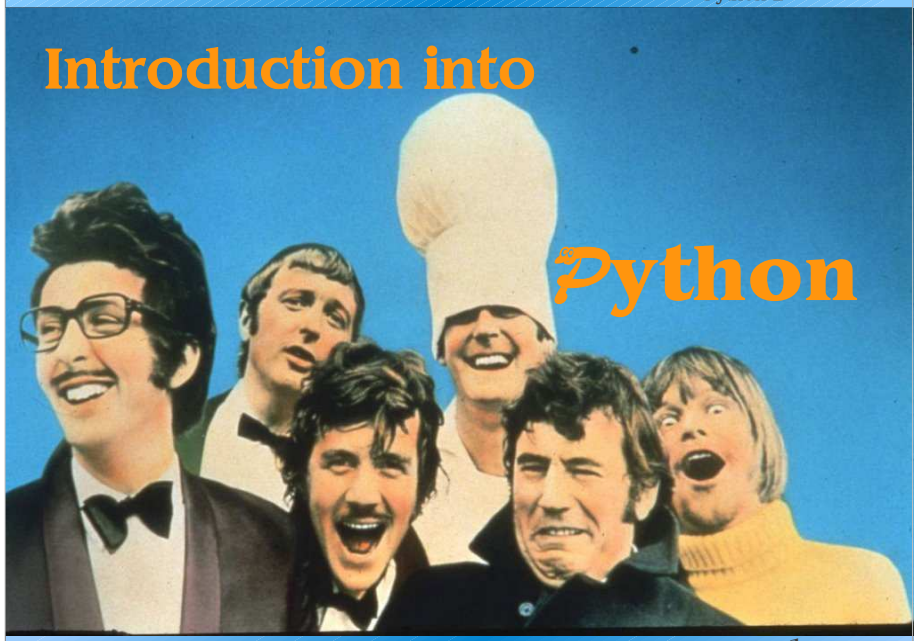
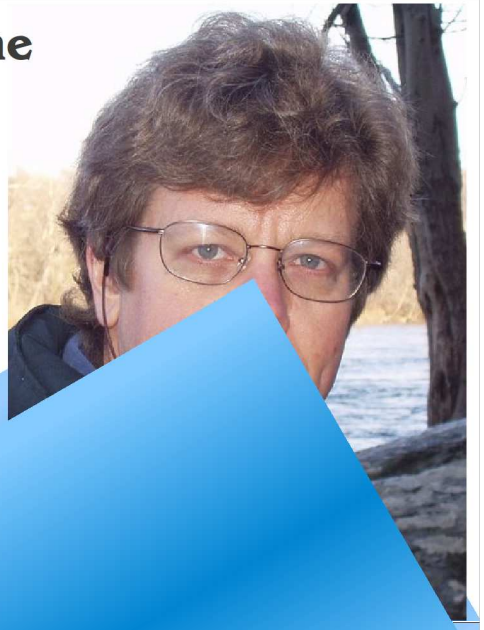
Python I

Introduction into Python



Origin of the Name

Python is fun to use and the this is reflected in the name:
Guido van Rossum, who designed Python in the 90ties in Amsterdam, named his language after the BBC-Series „Monty Python's Flying Circus“.



Programming Languages are like Shoes

Intentions of Python

Strongpoints of Python:

- simple but mighty
- modular programming
- easy to read
- Rapid Prototyping
- Other languages can easily be embedded
- Python can be easily extended
- Open Source
- Functional Programming

Python I

Python Installation

Python is contained in nearly every Linux distribution, e.g. Red Hat, OpenSuSE, Debian, Ubuntu, Mandriva etc.

Under Debian/Ubuntu:

```
apt-get install python
```

Under OpenSUSE:
YaST

There are two possibilities under Windows:

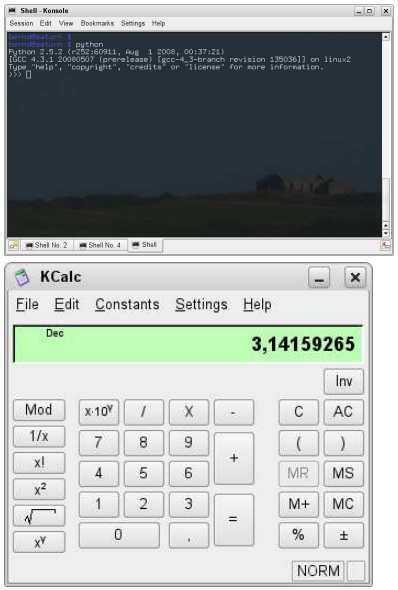
- The „official“ Python:
  http://www.python.org/ftp/python/
- Active Python
  It can be downloaded free of cost, but it not Open Source Software
  http://www.activestate.com/Products/ActivePython/

Starting Python

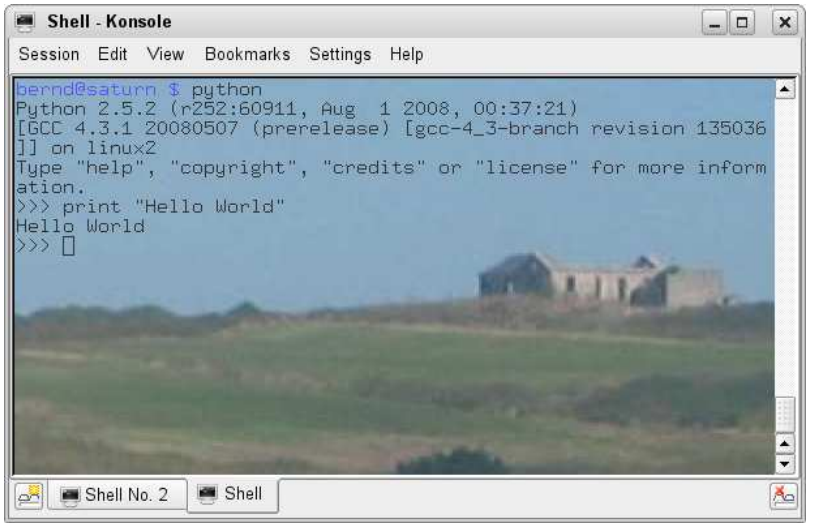
From the shell by starting "python" or "idle-python2.6"

In the Python shell any expression can be calculated:

```
>>> 3.14159256 * 4
12.566370239999999
>>> 5**4
625
>>> (1+1.045)**20
1636312.4956171759
>>>
```



The „mandatory“ Script



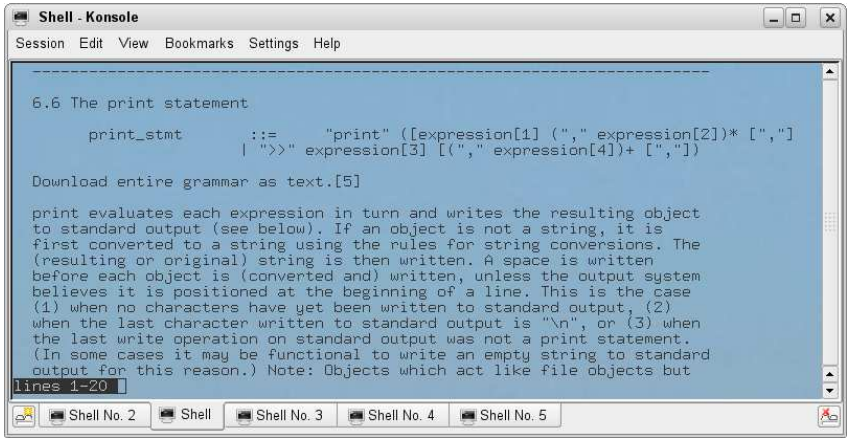
„Hello World“ in Python 3

The print statement has been turned into a function in Python3. That's why the arguments are in brackets:

```
$ python3.0
Python 3.0.1+ (r301:69556, Apr 15 2009,
17:25:52)
[GCC 4.3.3] on linux2
Type "help", "copyright", "credits" or
"license" for more information.
>>> print("Hello World!")
Hello World!
>>>
```

Using Help in the Interpreter

```
>>> help('print')
```



A comfortable Python Shell

IPython provides considerabl advantages over the “regular” Python shell (and maybe even bash):

- Syntax Highlighting
- Command Completion
- Definition of macros
- Tab completion
- Debugger (%pdp)

Installation of the Shell (Debian/Ubuntu):

sudo aptitude install ipython

Compiled or Interpreted?

Python is both an interpreted and a compiled language.

Python code is not compiled into machine language.

Python code is translated into intermediate code, which has to be executed by a virtual machine, known as the PVM, the Python virtual machine.

This is a similar approach to Java.

There is even a way of translating Python programs into Java byte code for the Java Virtual Machine (JVM).

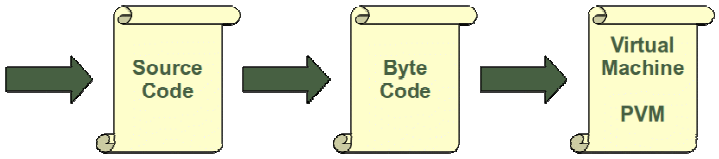
This can be achieved with Jython.

Internal Execution if Code

Every time a Python script is executed, byte code is created. But only, if a Python program is imported as a module, the byte code will be stored in the corresponding .pyc file:

```
>>> import lernen
Python lernen!
>>>
```

The Byte Code will be executed by the PVM.



Variables

A variable is “something” whose value changes over time.

A variable can be seen as a pigeonhole in memory which can be referred to by a nickname (identifier).

Declaration of variables is not required with Python.

If you need a variable, you just come up with a name and use it as a variable.

Valid Identifier Names

A valid identfier is a non-empty sequence of characters of any length with:

- the start character can be the underscore “_” or a capital or lower case letter
  in Python3: letters from non-English languages, like “Ä”, “ö”, are allowed
- the letters following the start character can be anything which is permitted as a start character plus the digits.
  in Python3: anything which Unicode considers to be a digit, non-whitespace characters.
- Identifiers are case-sensitive
- Python keywords are not allowed as identifier names

Python‘s Keywords

No identifier can have the same name as one of the Python keywords:

```
and       elif      import    raise
as        else      in        return
assert    except    is        try
break     finally   lambda    while
class     for       nonlocal  with
continue  from      not       yield
def       global    or
del       if        pass
```


Assignments

```
i = 42
```

The equal sign in the above statement shouldn't be mistaken for a mathematical equal sign.
The equal sign means: the value 42 will be assigned to the variable i, i.e. after the assignment the variable will have the value 42

```
>>> i = 42
>>> i = i + 1
>>> print(i)
43
>>>
```

No Declaration of Variables

Variables don't have to be declared.

There is a data type in Python, but it is bound to the value and not to the variable. This means, that the type of the variable may change during execution.

```
i = 42          # data type is integer(implicit)
i = 42 + 0.11   # type will be changed to float
i = "fourty"    # and now to a string
```


Casting in Python

A type can be explicitly changed from one to another type:

```
int()
float()
str()
```

There are situation, where this is necessary:

```
>>> x = 10
>>> y = 3
>>> x / y
3
>>> float(x) / y
3.3333333333333335
>>>
```

```
>>> x = "42"
>>> print int(x)
42
>>> x = '42.0'
>>> print float(x)
42.0
>>> print int(float(x))
42
>>> print int(x)
Traceback (most recent
call last):
  File "<stdin>", line 1,
in <module>
ValueError: invalid
literal for int() with
base 10: '42.0'
>>>
```


Number Data Types and Literals

- **Integers**
  normal Integers , like 42, 4321
  0 as a prefix: Octal number and 0x means Hex number
- **Long Integer**
  If an integer literal ends with an l or L, it becomes a Python long integer. It can grow as “long” as needed.
  Since Python 2.2. the letter “l” or “L” is no longer needed, as integers are automatically converted to long integers on overflow.
- **Floating-point numbers**
  numbers like 3.14159 or 17.3e+02
- **complex numbers**
  e.g. 1.2+3j

Floating-Point Numbers

The built-in float type holds double-precision floating point numbers.
The range depends on the compiler, Python was built with.
floats have limited precision!
This can be a problem, when comparing two floats.

```
>>> x = 1.0 % 0.1
>>> print(x)
0.1
>>>
```


The result should be 0.0!

General Decimal Arithmetic

Decimal floating point has finite precision with arbitrarily large bounds.

The purpose of this module is to support arithmetic using familiar "schoolhouse" rules and to avoid some of the tricky representation issues associated with binary floating point. The package is especially useful for financial applications or for contexts where users have expectations that are at odds with binary floating point

Decimal Module

```
>>> from decimal import *
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(0)
Decimal('Infinity')
>>> Decimal(1.0) / Decimal(0.1)
Decimal('10.0000000')
>>> Decimal(1.0) % Decimal(0.1)
Decimal('0.100000000')
```

Still a problem!

Strings

A String can be seen as a sequence of characters, which can be expressed in several ways:

- **single quotes** (')
  `'This a a string in single quotes'`
- **double quotes** (")
  `"Miller's dog bites"`
- **triple quotes**(''') **or** (""")
  `'''She said: "I don't mind, if Miller's dog bites"'''`

Fundamentals on Strings

- Concatenation
  `"Hello" + "World"  -> "HelloWorld"`
- Repetition
  `"*-*" * 3  -> "*-**-**-*"`
- Indexing
  `"Python"[0]  -> "P"`
- Slicing
  `"Python"[4:6]  -> "on"`
- Length
  `len("Python")  -> 6`

Escape-Characters in Strings

Esacpe Sequence	Meaning
\\	Backslash to keep a backslash
\'	Single quote
\"	Double quote
\a	Bell
\b	Backspace
\f	Formfeed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\N{id}	Unicode dbase id
\uhhhh	Unicode 16-bit hex
\Uhhhhhhhh	Unicode 32-bit hex
\xhh	Hex digits value hh
\ooo	Octal digits value
\0	Null (not like C the string termination)

print using Escape Character

```
>>> print "Just \tane\b idea"
Just    an idea
>>> print "Just \tane\b \ridea"
idea    an
>>>
```

The word “Just” has been overwritten!

```
>>> print 'Mr. Miller's cat'
  File "<stdin>", line 1
    print 'Mr. Miller's cat'
                        ^
SyntaxError: invalid syntax
>>> print 'Mr. Miller\'s cat'
Mr. Miller's cat
```


raw strings

Raw strings are processed without language specific interpretation.

In Python 'raw strings' are preceded by an r od R.

Backslashes in a raw string are not interpreted as escape sequences:

```
>>> x = r"In windows C:\Monty\Python"
>>> print x
In windows C:\Monty\Python
>>>
```


Strings are immutable

```
>>> str = "This is a Test"
>>> str[3]
's'
>>> str[3] = 'a'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item
assignment
```

Python I

Operators

`+, -`	Addition, Substraction	`10 - 3` → `7`
`*, /, %`	Multipl., Division, Remainder	`27 % 7` → `6`
`+x, -x`	(algebraic) sign	`-3 +3` → `0`
`~x`	Bitwise Not	`~3` → `-4`
	`(~ x` corresponds to `-(x-1))`	
`**`	Exponentiation	`10 ** 3` → `1000`
`or`	Boolean Or	
`and`	Boolean And	
`not`	Boolean Not	
`in`	„Element of“	`1 in [3, 2, 1]`
`<, <=, >, >=, !=, ==`	Comparisons	`2 <= 3`
`|`	Bitwise Or	`6 | 3` → `7`
`&`	Bitwise And	`6 & 3` → `2`
`^`	Bitwise XOR	`6 ^ 3` → `5`
`<<, >>`	Shift Operators	`2 << 3` → `16`

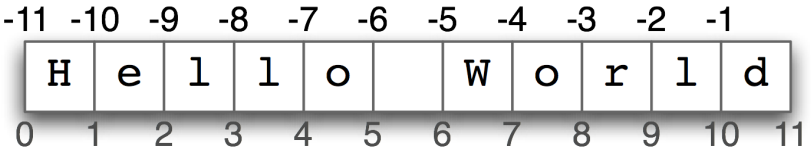
Comprehension Test

```
number    = 19
width     = 4.4
separator = ';'
```

What is the result of the following expressions?

1) number / 2
2) width / 2
3) number / 2.0
4) separator * 3
5) separator * width

Indexing



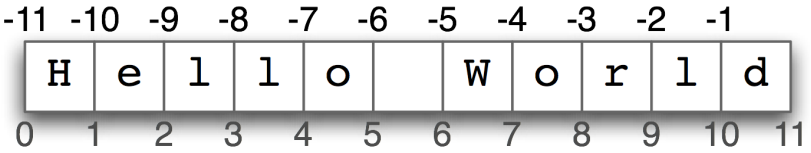
Particular Elements of a String (characters) can be indexed and accessed with square brackets:

```
>>> txt = "Hello World"
>>> txt[0]
'H'
>>> txt[4]
'o'
```

Indices can be accessed from backwards as well:

```
>>> txt[-1]
'd'
>>> txt[-5]
'W'
```


Slicing




```
>>> txt = "Hello World"
>>> txt[1:5]
'ello'
>>> txt[0:5]
'Hello'
>>> txt = "Hello World"
>>> txt[0:5]
'Hello'
```

```
'Hello'
>>> txt[0:-6]
'Hello'
>>> txt[:5]
'Hello'
>>> txt[6:]
'World'
```

Lists

Lists can contain arbitrary Python-Objects:

```
>>> colours = ['red', 'green', 'blue']
>>> colours[0]
'red'
>>> colours
['red', 'green', 'blue']
>>>
>>>
>>> a = ["Swen", 45, 3.54, "Basel"]
>>> a[3]
'Basel'
>>>
```


Appending New Elements

An element can be appended to a list L with the "+" operator

```
L = L + [item]
```

or with the append method:

```
L.append(item)
```

Appending a list to a list:

```
>>> list1 = ["Monty", "Pyton"]
>>> list2 = ["Faulty", "Towers"]
>>> list1.extend(list2)
>>> print list1
['Monty', 'Pyton', 'Faulty', 'Towers']
```

Inserting items at certain positions:

```
L.insert(index,item)
```

alternatively:

```
L = L[:index] + [item] + L[index:]
```


Lists

Lists can contain other lists (sublists):

```
>>> pers = [["Marc","Mayer"],["Hauptstr. 17",
"12345","Musterstadt"],"07876/7876"]
>>> name = pers[0]
>>> name[1]
'Mayer'
>>> adresse = pers[1]
>>> adresse[1]
'12345'
>>> pers[2]
'07876/7876'
>>> strasse = pers[1][0]
>>> strasse
'Hauptstr. 17'
>>>
```


Slicing a List

A "slice" of a list can be created by specifying two indices. The return value is a new list containing all the elements of the list, in order, starting with the first slice index up to but not including the second slice index:

```
>>> colours = ['red', 'green', 'blue']
>>> colours[1:3]
['green', 'blue']
>>> colours[2:]
['blue']
>>> colours[:2]
['red', 'green']
>>>
>>> colours[-1]
'blue'
```

Sequential Data Types

Sequential Data Types are sequences of elements consist of elements which have a defined order.

Such as

- **str**
- **unicode**
- **list** (mutable)
- **tuple** (like lists, but immutable)

Operators for sequential data types

`x in s`	True, if x is contained in s, otherwise False
`x not in s`	True, if x is not contained in s, otherwise False
`s + t`	Returns a new sequence, which contains the concatenation of s and t.
`s += t`	similar, but the result will be assigned to s
`s * n or n * s`	Result is a new Sequence, which contains the of n copies of s.
`s *= n`	Like the previous one, but the result will be assigned to s

Operators for sequential data types

`s[i]`	i'th item of s.
`s[i:j]`	slice of s from i to j.
`s[i:j:k]`	slice of s from i to j with step k
`len(s)`	number of items of s
`min(s)`	smallest item of s, if a complete order is defined
`max(s)`	largest item of s, if a complete order is defined

Examples

```
>>> s = "Just a string"
>>> "s" in s
True
>>> "s" not in s
False
>>> l = [1,2,4,9,16,25]
>>> 2 in l
True
>>> 3 in l
False
>>> len(l)
6
>>> len(s)
14
```

Further Examples

```
>>> 2 * list
[1, 2, 4, 9, 16, 25, 1, 2, 4, 9, 16, 25]
>>> str * 2
'Just a stringJust a string'
>>> str
'Just a string'
>>> min(str)
' '
>>> min(list)
1
>>> min("Python")
'P'
>>> max("Python")
'y'
```

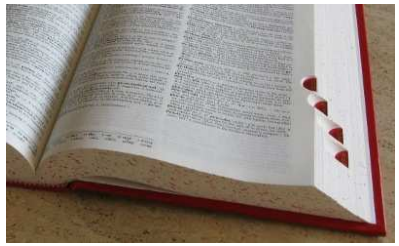

Further Operators for Lists

`s[i:j] = t`	`s[i:j]` will be replaced by t
`s[i:j:k] = t`	the elements of `s[i:j:k]` will be replaced by t
`del s[i]`	The i'te of s will be removed
`del s[i:j]`	The elements `s[i:j]` will be removed from s. (equivalent to `s[i:j] = []`).
`del s[i:j:k]`	The elements of `s[i:j:k]` will be removed from s.

Examples

```
>>> list = [1,2,4,9,16,25]
>>> list = [1,2,3,4,5,6,7,8,9,10]
>>> sq = [1,2,4,9,16]
>>> list[0::2]
[1, 3, 5, 7, 9]
>>> list[1::2]
[2, 4, 6, 8, 10]
>>> list[1::2]=sq
>>> list
[1, 1, 3, 2, 5, 4, 7, 9, 9, 16]
>>> del list[0::2]
>>> list
[1, 2, 4, 9, 16]
```


Dictionaries



Dictionaries are sometimes found in other languages as “associative arrays” or “hashes” in Perl.

Dictionaries can be thought of as unordered sets of (key,value)-pairs.

Unlike sequences, which are indexed by a range of numbers, dictionaries are indexed by keys, which can be any immutable type, i.e. strings and numbers can always be keys.

Tuples can be used as keys if they contain only strings, numbers, or tuples.

If a tuple contains any mutable object either directly or indirectly, it cannot be used as a key.

Example

```
>>> tel = {'moe':'0000', 'ned':'8904',
'homer':'3223'}
>>> tel
{'homer': '3223', 'ned': '8904', 'moe': '0000'}
>>> tel['moe']
'0000'
>>>
```

Dictionaries can be incrementally created:

```
>>> numbers = {}
>>> numbers["one"] = "eins"
>>> numbers["two"] = "zwei"
>>> numbers["three"] = "drei"
>>> print numbers
{'three': 'drei', 'two': 'zwei', 'one': 'eins'}
>>>
```


Dictionaries

The values of a dictionary can be arbitrary data types.

The keys can only be instances of immutable data types,
e.g. integers, floats, strings, and tuples but no dictionaries or lists.

```
>>> dic = { [1,2,3]:"abc"}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: list objects are unhashable
```

Aber Tupel als Schlüssel sind okay:

```
>>> dic = { (1,2,3):"abc", 3.1415:"abc"}
>>> dic
{(1, 2, 3): 'abc'}
```


Operators on Dictionaries

`len(d)` number of elements, i.e. (key, value)-pairs

```
>>> print numbers
{'three': 'drei', 'two': 'zwei', 'one': 'eins'}
>>> print len(numbers)
3
```

`del d['k']` delete the k'th element

```
>>> del numbers['two']
>>> print numbers
{'three': 'drei', 'one': 'eins'}
>>>
```


Operators on Dictionaries

`k in d` True, if there is a key `k` in `d`

```
>>> "one" in numbers
True
>>> "two" in numbers
False
>>>
```

`k not in d` True, if there is no key `k` in `d`

```
>>> "one" not in numbers
False
>>> "two" not in numbers
True
>>>
```

Accessing non existing keys

If you try to access a key, which doesn't exist, an error will be thrown:

```
>>> woerter = {"house" : "Haus", "cat":"Katze"}
>>> woerter["car"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'car'
```

A better way to do it:

```
>>> if "car" in woerter: print woerter["car"]
...
>>> if "cat" in woerter: print woerter["cat"]
...
Katze
```


Dictionaries

```
>>> print numbers
{'three': 'drei', 'two': 'zwei', 'one':
'eins'}
>>> x = numbers
```

You could assume, that the data of numbers will be copied and saved under the name x.
This isn't true. x points to numbers.
This means: Whatever you do to x you do to numbers as well:

```
>>> del x["one"]
>>> print numbers
{'three': 'drei', 'two': 'zwei'}
>>>
```


copy() and clear()

The methods copy() can be used to copy a dictionary:

```
>>> w = woerter.copy()
>>> woerter["cat"]="chat"
>>> print w
{'house': 'Haus', 'cat': 'Katze'}
>>> print woerter
{'house': 'Haus', 'cat': 'chat'}
```

clear() empties the content of a dictionary. The dictionary lingers on as an empty dictionary:

```
>>> w.clear()
>>> print w
{}
```


Converting Dictionaries in Lists

```
>>> w={"house":"Haus","cat":"Katze","red":"rot"}
>>> w.items()
[('house', 'Haus'), ('red', 'rot'), ('cat',
'Katze')]
>>> w.keys()
['house', 'red', 'cat']
>>> w.values()
['Haus', 'rot', 'Katze']
```

Zipping Lists

```
>>> x = [1, 2, 3]
>>> y = [4, 5, 6]
>>> z = zip(x, y)
>>> z
[(1, 4), (2, 5), (3, 6)]
>>> x2, y2 = zip(*z)
>>> print x2
(1, 2, 3)
>>> print y2
(4, 5, 6)
>>> x == list(x2)
True
```


Converting Lists in Dictionaries

Two lists are given: One contains the keys, the other one the values on corresponding positions.
Task: Turning these lists into a dictionary

```
>>> dishes =
["Pizza","Sauerkraut","Paella","Frogs"]
>>> countries =
["Italy","Germany","Spain","France"]
>>> prejudices = zip(countries, dishes)
>>> print prejudices
[('Italy', 'Pizza'), ('Germany', 'Sauerkraut'),
('Spain', 'Paella'), ('France', 'Frogs')]
>>> prej_dict = dict(prejudices)
>>> print prej_dict
{'Germany': 'Sauerkraut', 'Italy': 'Pizza',
'Spain': 'Paella', 'France': 'Frogs'}
```


Dictionaries concatenating

Method update() can be used to append a dictionary into another dictionary. If the second dictionary contains keys, which are contained in the first one as well, they get reassigned with the values of the second dictionary.

```
>>> w={"house":"Haus","cat":"Katze","red":"rot"}
>>> w1 = {"red":"rouge","blau":"bleu"}
>>> w.update(w1)
>>> print w
{'house': 'Haus', 'blau': 'bleu', 'red':
'rouge', 'cat': 'Katze'}
```

Question:

```
w1.update(w) == w.update(w1)
```


Interactive Help

If you need help, say so,
or better type it!


```
$ python
Python 2.6.5 (r265:79063, Apr 16 2010, 13:57:41)
[GCC 4.4.3] on linux2
Type "help", "copyright", "credits" or "license"
for more information.
>>> help
Type help() for interactive help, or help(object)
for help about object.
>>>
```

You need parenthesis, i.e. help()

help()

```
>>> help
Type help() for interactive help, or help(object) for help about object.
>>> help()

Welcome to Python 2.6!  This is the online help utility.

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at http://docs.python.org/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, or topics, type "modules",
"keywords", or "topics".  Each module also comes with a one-line summary
of what it does; to list the modules whose summaries contain a given word
such as "spam", type "modules spam".

help>
```


Interactive Help

There is help for the following topics:

- Records
- Modules and
- special topics

Call:

```
>>> help()
```

and then after the prompt

```
help> topics
```

or

```
help> keywords
help> modules
```


Help Concerning Methods

While in the Python Interpretor (not in the Help Mode) use help with the function or method name as a parameter:

```
>>> help(len)
```

The result will be:

```
Help on built-in function len in module
__builtin__:

len(...)
    len(object) -> integer

    Return the number of items of a sequence or
mapping.
(END)
```

Use „q“ to leave this mode!

Have a Go!

Something to try out:

```
help(dict)
help(list)
help(dict.pop)
help(list.extend)
help(dict.copy)
```

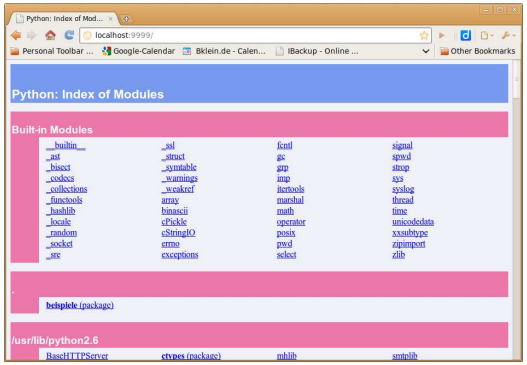
Python Docus via Webbrowser

Starting the Server:

```
$ pydoc -p 9999
pydoc server ready at http://localhost:9999/
```

The documentation can be browsed under the URL

localhost:9999

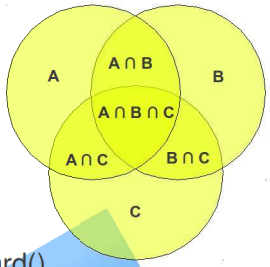


Sets

Sets are unordered collections of unique elements.

Various operations can be executed on sets:

- membership testing (x in set, x not in set),
- remove an element from a set, i.e. set.discard()
- intersection, i.e. set.intersection()
- union, i.e. set.union()
- difference, i.e. set.difference()
- symmetric difference, i.e. set.symmetric_difference()
  the symmetric difference are all elements which are in exactly one of the sets.



Sets

A set can be created iteratively:

```
>>> s = set()
>>> s.add("a")
>>> s.add("b")
>>> s.add("c")
>>> print(s)
set(['a', 'c', 'b'])
```

or directly while being initialized:

```
>>> s = set(["a","b","c"])
>>> print(s)
set(['a', 'c', 'b'])
```


Further Operations on Sets

```
>>> A = set((3,78,43,2,9))
>>> B = set((2,89,7,9,78))
>>> C = set((89,9,3,8))
>>> A | B
set([2, 3, 7, 9, 43, 78, 89])
>>> A & B
set([9, 2, 78])
>>> A & B & C
set([9])
>>> A | B & C
set([2, 3, 89, 9, 43, 78])
>>> (A | B) & C
set([89, 3, 9])
>>> A ^ B # symmetric difference
set([3, 7, 43, 89])
>>> A - B # difference
set([3, 43])
```

Subset, Superset

```
>>> E = set(range(2,100,2))
>>> O = set(range(1,100,2))
>>> A = set(range(1,100))
>>> E <= A
True
>>> A >= O
True
>>> E | O == A
True
>>> E & O == set()
True
```


Sets in Python3

It's possible to write sets with curly braces in Python 3 and Python 2.7.
So, the following two notations are equivalent:

```
s = set([42,47,11])
```
and
```
s = {42,47,11}
```

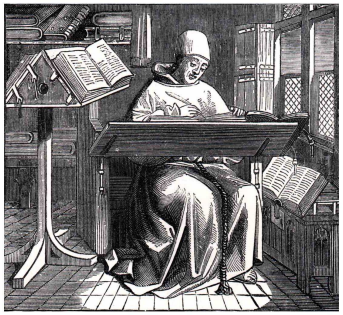
Sets can be seen as valueless dictionaries, though the type is different:

```
>>> s = {42,47,11}
>>> type(s)
<class 'set'>
>>> type({})
<class 'dict'>
```


Writing Scripts

So far we have just typed in single commands in the interpretor.

What about writing scripts?



Python Script Files

Python Programs are usually stored in files.

Python source files are saved with the suffix .py.

The character combination `#!` at the beginning of a script is called **shebang** or **hashbang**.

The Shebang line consists of the shebang, the name of the interpreter and the full path of the interpretor.

„Hello World“ as Python File

The file:

```
#!/usr/bin/python
print "Hello World"
```

make it executable:

```
chmod 755 hello.py
```

execute:

```
$ ./hello.py
Hello World
$
```

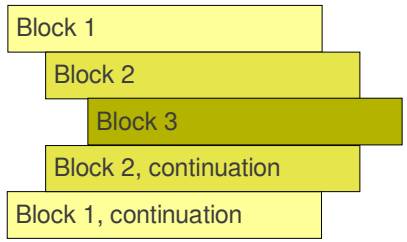
Use `#!/usr/bin/env python` instead of `!/usr/bin/python`

Grouping Statements

Most programming languages use certain characters or keywords to group statements:

- `begin ... end`
- `do ... done`
- `{ ... }`
- `if ... fi`

Python's block principle:



Indentation

Python uses indentation for structuring the code.

All statements with the same distance to the left belong to the same block of code, i.e. the statements within a block line up vertically.
The block ends at a line less indented or the end of the file.
If a block has to be more deeply nested, it is simply indented further to the right.

A statement can be continued on the next line with the continuation character „\“.

if statement

Any script needs a way to branch the course of execution in certain situations. The if statement is a compound statement, i.e. it may contain other statements including other if's.

The if statement is a way to make sure that parts of the code will only be executed under certain conditions.

if Statement

```
if condition:
    # statements
else:
    # statements
```

If the condition following the if is evaluated to True, the first nested block will be executed. In case of False the second nested block will be executed, i.e. the one following the else.

if-Statement

```
if condition1:
    # statements
elif condition2:
    # statements
else:
    # statements
```

The else-part of the if statement may contain a nested if statement. In this case we can write "`elif`" instead of "`else if`"

Conditions

```
>>> 1 == 1
True
>>> 1 == 1.0
True
>>> 1 == 1.3
False
>>> a = 3
>>> b = 3
>>> a == b
True
>>> c1 =["A", "B", "C"]
>>> c2 =["A", "B", "C"]
>>> c1 == c2
True
>>>
```

Readin User Input

The easiest way to read a line from input is the function raw_input():

```
raw_input(prompt)
```

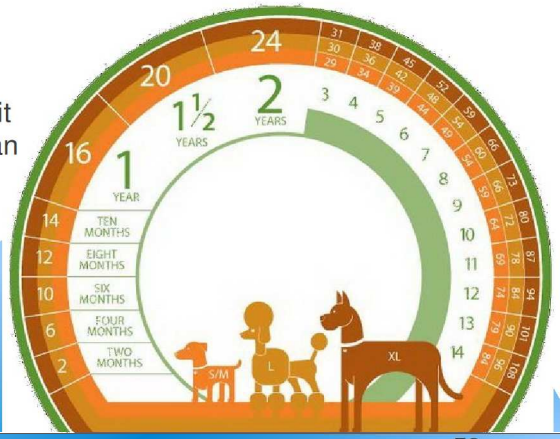
It returns to the script the line of text from standard input read as a string.
Optionally, it accepts a string that will be printed as a prompt

```
name = raw_input("What's your name: ")
print "Your name is " + name
```


Dog Years

It is a common belief that 1 human year is equal to 7 dog years. That is not very accurate, since it's not a linear relation.

It's better to assume:
If a dog is one year old, it corresponds to 14 human years. If a dog is two years of age, it corresponds to 22 human years. After this every year corresponds to five human years.



Solution

```
age = input("Dog's age: ")
print
if age < 0:
   print "This can't be true!"
elif age == 1:
   print "about 14 years"
elif age == 2:
   print "about 22 years"
elif age > 2:
   human = 22 + (age -2)*5
   print "in human years:", human

###
raw_input('press Return>')
```


Exercise

Write a program, which contains a dictionary with usernames and corresponding passwords.

The script shall prompt for a username and a password.

The script checks if this username and password is an element of the dictionary.

Solution

```
pws = {"user1":"secret1", "user2":"secret2"}

user = raw_input("user: ")
password = raw_input("password: ")

okay = False
if user in pws:
    okay = (pws[user] == password)

if (okay):
    print "Okay"
else:
    print "password and/or user unknown"
```


True or False

False are:

- numerical Null Values (`0, 0L, 0.0, 0.0+0.0j`),
- the boolean value **`False`**,
- empty strings,
- empty lists,
- empty tuples,
- empty dictionaries.
- and the special value None.

All other values are considered to be "true".

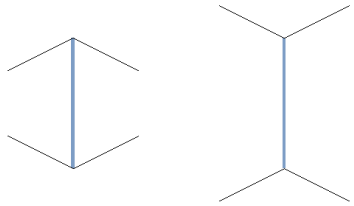
'Abbreviated' if, Example

The following construct is well-known to any C programmer:

```
max = (a > b) ? a : b;
```

It's an abbreviation of:

```
if (a > b)
   max=a;
else
   max=b;
```



'Abbreviated' if, Example

The Python way:

```
max = a if (a > b) else b
```

instead of the C way:

```
max = (a > b) ? a : b;
```

as an abbreviation of:

```
if a > b:
   max=a
else:
   max=b
```

Python Modules

Modules usually correspond to Python program files or C extensions, i.e. each Python file is a module.

Usage of existing modules:
If you want to use a module, the have to be included with import:

```
import math
from math import sin, cos
```
or
```
from math import *
```


Command Line Parameters

The sys modul contains a list with the command line parameters:
`sys.argv`

The command line arguments can be printed liks this:

```
import sys
print sys.argv
```

If those two lines are saved in par_test.py, we can see the following:

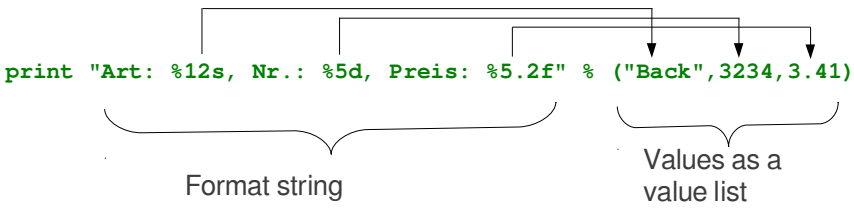
```
$ par_test.py erstes zweites drittes Argument
['./par_test.py', 'erstes', 'zweites',
'drittes', 'Argument']
```


String Formatting

% is the remainder operator for integer numbers, but in string context it is overloaded to work on strings.
The % operator provides a simple way to format values as strings, according to a format definition string. (similar to C's sprintf function)

The way it works:
1. A format string is positioned on the left of the % operator with embedded conversion targets starting with a % (e.g., "%5d").
2. On the right side of the % operator is a tuple of values, each corresponding to the conversion targets on the left side.

Formatted output with print



`%12s`	is a placeholder for a string literal or string variable
`%5d`	an integer with 5 digits
`%5.2f`	floating point number
`%-5.2f`	floating point number

Alternatively: str.format()

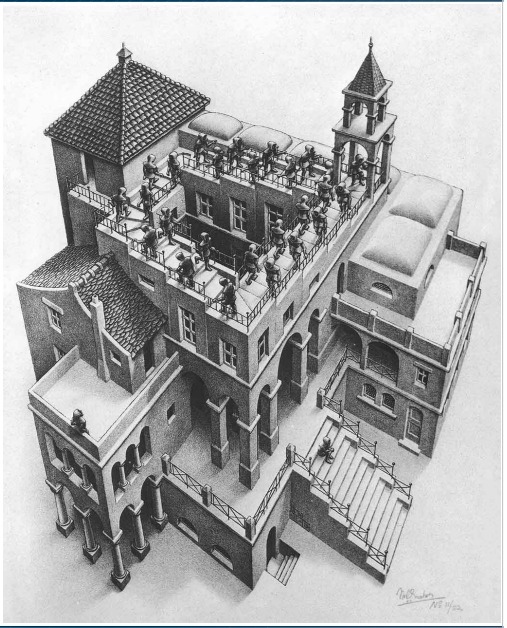
```
>>> a = 12.3
>>> b = 4.5687
>>> print "length: %6.2f, width: %6.2f" % (a,b)
length:  12.30, width:   4.57
>>> print "length: {0}, width: {1}".format(a,b)
length: 12.3, width: 4.5687
>>> print "length: {0:6.2f}, width:
{1:6.2f}".format(a,b)
length:  12.30, width:   4.57
>>>
```


Loops

Most programming languages need loops to repeat the execution of a statement block while some condition is true.
Python has two kinds of loops:

- while
- for

Implicit loops (iterations): the map, reduce, and filter functions and others



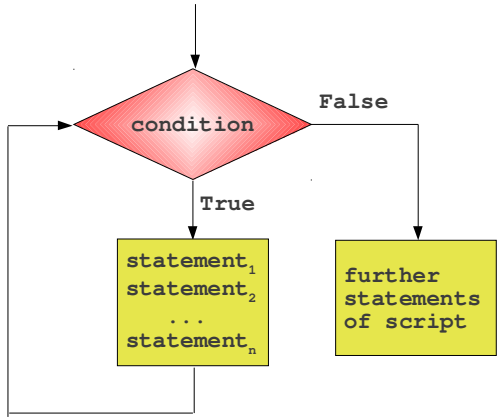
while Loop

The while statement is the most general iteration construct of Python.
It repeatedly executes a block of indented statements, “while” the test at the top is evaluating to a true value.
When the test becomes false, control continues after the indented block.
The statements of the additional else branch will be executed, if the loop body of the while loop hasn't been left by break.

```
while <test>: # Loop test
   # statements of loop body
else: # Optional else
   # Run if while hasn't been left by break
```


While Loop: Syntax

```
while condition:
   statement1
   statement2
   ...
   statementn
```



Example: Sum of 1 to n

```
#!/usr/bin/env python

n = 100

sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i + 1

print "Sum of 1 to %d: %d" % (n,sum)
```


Exercise

Write a program, which asks for the initial balance K_0 and for the interest rate.
The program shall calculate the new capital K_1 after one year including the interest.

Extend the program with a while-loop, so that the capital K_n after a period of n years can be calculated.

Solution

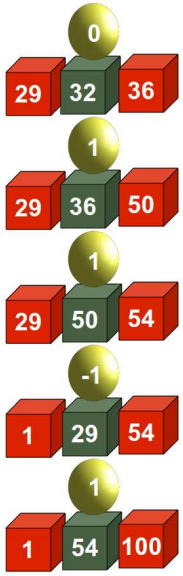
```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

K = float(raw_input("Starting capital? "))
p = float(raw_input("Interest rate? "))
n = int(raw_input("Number of years? "))

i = 0
while (i < n):
    K = K + K * p / 100.0
    i = i + 1
    print i
print "Capital after " + str(n) + " ys: " + str(K)
```


Guessing Number Game

Write a program, which lets a human player guess a number between a number range from 1 to n. The player inputs his gues. The program informs the player, if this number is larger, smaller or equal to the secret number, i.e. the number which the program has randomly created.

Hint: It's possible to create a random number between 1 and 20 with the following code:

```
import random
n = 20
to_be_guessed = random.randint(1,n)
```



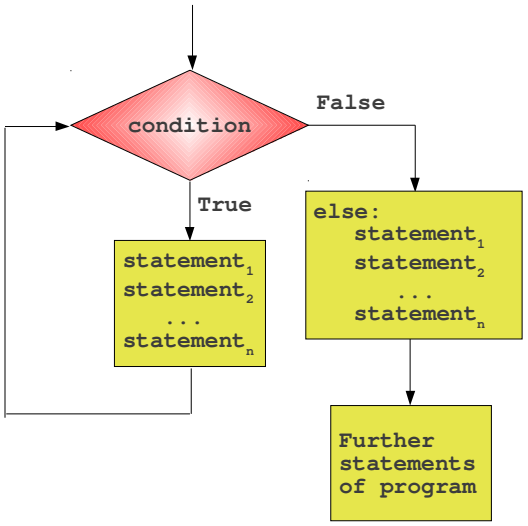
Solution:

```
import random
n = 20
to_be_guessed = random.randint(1,n)
guess = 0
while guess != to_be_guessed:
    if guess > 0:
        if guess > to_be_guessed:
            print "Number is too large"
        else:
            print "Number is too small"
    guess = input("New guess: ")
print "You got it!"
```


The else Branch

The else branch of the while construct is something special in Python, which is unknown in other programming languages.

```
while condition:
   statement1
   ...
   statementn
else:
   statement1
   ...
   statementn
```



New Version of the Game

Game with else branch:

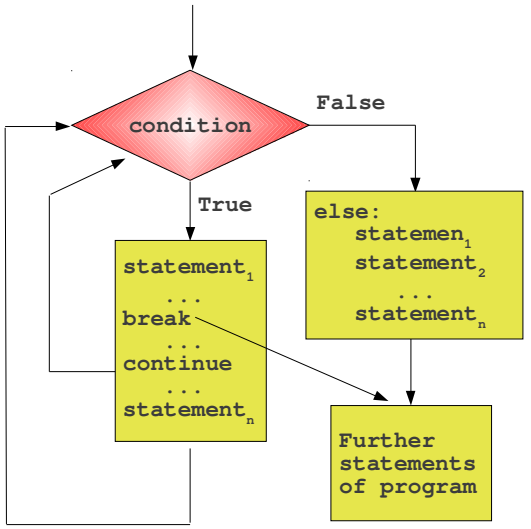
```
import random
n = 20
to_be_guessed = int(n * random.random()) + 1

guess = 0
while guess != to_be_guessed:
    if guess > 0:
        if guess > to_be_guessed:
            print "Number too large"
        else:
            print "Number too small"
    guess = input("New guess: ")
else:
    print "You got it!"
```

But it's no improvement at all :-(

The else Branch

Normally, the while finishes, if the condition is not fulfilled anymore.

Additionally while loop can be exited by the unconditional break statement.
The else branch of the while construct will not be executed, if the while is left via break.



Exercise

Extend our guessing game to allow a give-up.

If the human player inputs a 0 or a negative number, he is giving up.

Implement this behaviour by using the else branch with the while loop

New Version With break

```
import random
n = 20
to_be_guessed = int(n * random.random()) + 1
guess = 0
while guess != to_be_guessed:
    guess = input("New number: ")
    if guess > 0:
        if guess > to_be_guessed:
            print "Number too large"
        else:
            print "Number too small"
    else:
        print "Sorry, that you're giving up!"
        break
else:
    print "Congratulation. You made it!"
```


For Loop

```
for variable in sequence:
   statement1
   statement2
   ...
   statementn
else:
   statement1
   statement2
   ...
   statementm
```

The for loop is a generic sequence iterator:
It can step through the items of any ordered sequence object.
The for loop works on

- strings,
- lists,
- tuples, and
- other iterators

Iterating over Strings

```
>>> for c in "Python":
...     print c
...
P
y
t
h
o
n
>>>
```

Iterating over Lists

```
fib = [0,1,1,2,3,5,8,13,21]
for i in fib:
    print i,
```

If you have to access the indices(), it can be achieved by using range() on the length of the list:

```
fib = [0,1,1,2,3,5,8,13,21]
for i in xrange(len(fib)):
    print i,fib[i]
print
```


Iterate through a Dictionary

No method is needed to iterate over the keys:

```
for key in d:
   print key
```

Alternatively, the method iterkeys() can be used:

```
for key in d.iterkeys():
   print key
```

itervalues() can be used to iterate over the values:

```
for val in d.itervalues():
   print val
```

Alternatively, it can be done like this:

```
for key in d:
   print d[key]
```


The range() Function

Programmers of other programming languages like C or C++ are used to a different kind of for loops:

```
for (i = 0; i < n;  i++) printf("%d, ",i);
```

Such a behaviour can be imitated in Python with the help of the range() function:

- `range(stop)`
- `range(start,stop)`
- `range(start,stop,step)`

```
>>> range(5)
[0, 1, 2, 3, 4]
>>> range(3,7)
[3, 4, 5, 6]
>>> range(3,20,4)
[3, 7, 11, 15, 19]
```


Sum of Numbers with for

The sum of the numbers from 1 to 100 can be calculated like this:

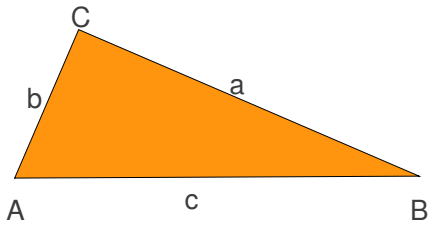
```
n = 100
sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i + 1
print sum
```

equivalent

```
n = 100
sum = 0
for i in range(1,n+1):
    sum = sum + i
print(sum)
```

Exercise: Pythagorean Numbers

Three natural number **a**, **b** and **c** satisfying
$a^2+b^2=c^2$
are called
pythagorean numbers.



Write a program, which prints all the pythagorean numbers smaller than a given number **n**.

Script: Pythagorean Numbers

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from math import sqrt

n = raw_input("maximal number? ")

n = int(n)+1
for a in range(1,n):
    for b in range(a,n):
        c_square = a**2 + b**2
        c = int(sqrt(c_square))
        if ((c_square - c**2) == 0):
            print a, b, c
```


Iterating over Lists

```
colours = ["red"]
for i in colours:
    if i == "red":
        colours += ["black"]
    if i == "black":
        colours += ["white"]
print colours
```

→ `['red', 'black', 'white']`

```
colours = ["red"]
for i in colours[:]:
    if i == "red":
        colours += ["black"]
    if i == "black":
        colours += ["white"]
print colours
```

→ `['red', 'black']`

The pass Statement

While writing a script it often happens, that you don't want to code some part of the script, e.g. the else part of an if loop. You can "mark" it for later completion with the **pass** statement.

```
x = 5
if ( x == 1):
else:
    print x
```

→

```
  File "pass.py", line 6
    else:
       ^
IndentationError: expected
an indented block
```

Solution with pass:

```
x = 1
if ( x == 1):
    pass
else:
    print x
```

type()

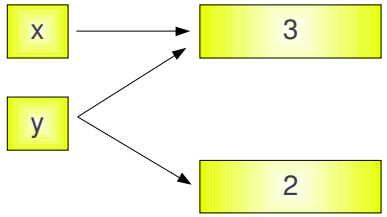
type() can be used to determine the data type of a variable:

```
l = [(1,2),"Python",["a",1],1,1.3]
for x in l:
    if type(x) == tuple:
        print x, " is a Tuple"
    if type(x).__name__ == 'list':
        print x, " is a List"
    if type(x).__name__ == 'str':
        print x, " is a String"
    if type(x).__name__ == 'int':
        print x, " is an integer"
    if type(x).__name__ == 'float':
        print x, " is a float"
```


'Strange' Behaviour

Variables in Python behave differently than those in C or C++

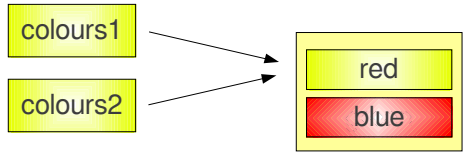
```
>>> x = 3
>>> y = x
>>> y = 2
>>>
```



Lists and Assignments

Lists are not copied but referenced:

```
>>> colours1 = ["red","green"]
>>> colours2 = colours1
>>> colours2[1] = "blue"
>>> colours1
['red', 'blue']
```



Augmented Assignments

Python provides another interesting feature, borrowed from the programming language C, the so-called augmented assignment (also called “in place operators”):
+=, -=, *= and so on

The following two forms are (nearly) equivalent:

```
x = x + y
x += y
```

In the second case the left side has to be evaluated only once. Augment assignments may be applied for mutable objects as an optimization.

Append, „+“ and augmented assignment

```
>>> x = [3,8,7]
>>> y = x
>>> y.append(9)
>>> print y
[3, 8, 7, 9]
>>> print x
[3, 8, 7, 9]
>>> y = y + [10]
>>> print x
[3, 8, 7, 9]
>>> print y
[3, 8, 7, 9, 10]
```

but:

```
>>> x = [3,8,7]
>>> y = x
>>> y += [10]
>>> print x
[3, 8, 7, 10]
>>> print y
[3, 8, 7, 10]
```


Identity

`id(object)`

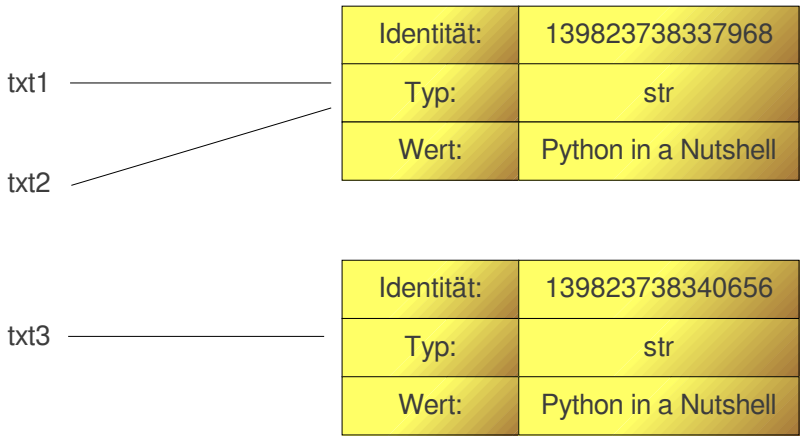
Returns the “identity” of an object. It is an integer (long integer), guaranteed to be unique and constant for the lifetime of this object.

Two objects with non-overlapping lifetimes may have the same `id()` value.

Example id()

```
>>> txt1 = "Python"
>>> txt3 = "Python"
>>> txt2 = txt1
>>> id(txt1) == id(txt2)
True
>>> id(txt1) == id(txt3)
True
>>> txt1 = "Python in a Nutshell"
>>> txt3 = "Python in a Nutshell"
>>> txt2 = txt1
>>> id(txt1) == id(txt3)
False
>>> id(txt1) == id(txt2)
True
```

3 References, 2 Instances



Deep Copy

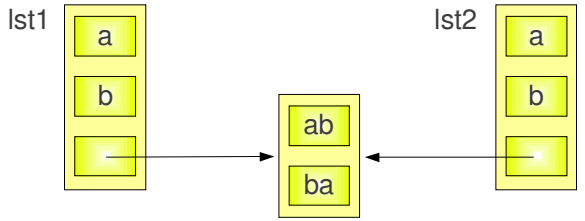
```
>>> lst1 = ['a','b',['ab','ba']]
>>> lst2 = lst1
>>> lst2[0] = 'c'
>>> print lst1
['c', 'b', ['ab', 'ba']]
>>> lst1 = ['a','b',['ab','ba']]
>>> lst2 = lst1[:]
>>> lst2[0] = 'c'
>>> print lst1
['a', 'b', ['ab', 'ba']]
>>> lst2[2][0] = 'c'
>>> print lst1
['a', 'b', ['c', 'ba']]
```

Problem:
lst1[:] creates only a shallow copy of lst1

Copying with [:]

Let's look at the following statements:

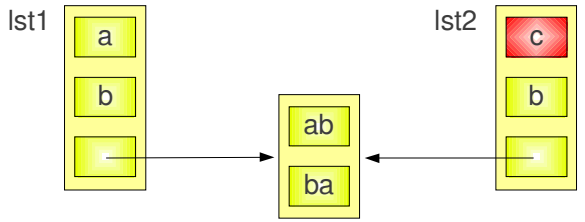
```
>>> lst1 = ['a','b',['ab','ba']]
>>> lst2 = lst1[:]
```



Copying with [:]

After execution of:

```
>>> lst2[0] = 'c'
```

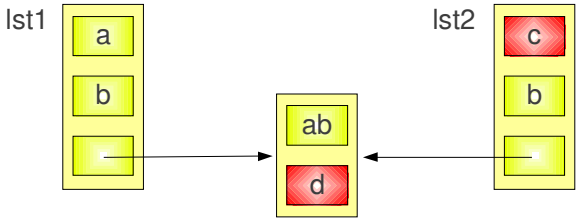


You can see that lst1 is still unchanged.

Copying with [:]

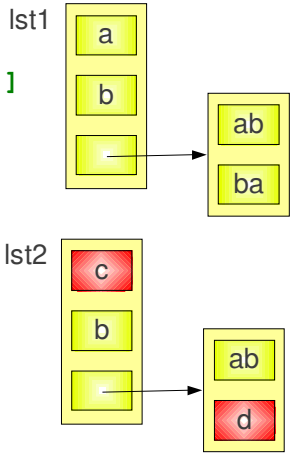
A problem exists with: `lst2[2][1] = 'd'`

```
>>>
```



The 'real' deepcopy

```
>>> from copy import deepcopy
>>> lst1 = ['a','b',['ab','ba']]
>>> lst2 = deepcopy(lst1)
>>> lst2[0] = "c"
>>> lst2[2][1] = "d"
>>> lst1
['a', 'b', ['ab', 'ba']]
>>> lst2
['c', 'b', ['ab', 'd']]
>>>
```

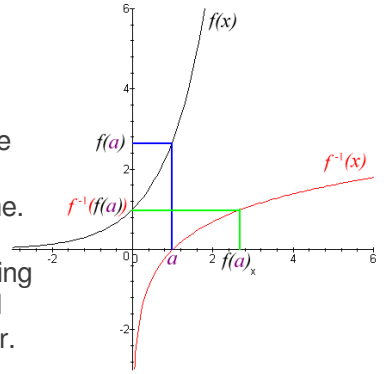


Functions

A Python function consists of Python code, that can be called repeatedly, with different inputs and outputs each time.

The benefit of functions:

- Code reuse
  Functions are an easy way to package logic so you are able to use it in more than one place and more than one time.
- Procedural decomposition
  Functions also provide a tool for splitting systems into pieces with well-defined behaviour, which keeps things simpler.



Functions

The general format is as follows:

```
def <name>(<Parameter List>):
   <statements>
```

The `<Parameter List>` consists of one or more variables, which are separated by commas:

```
arg1, arg2,... argN
```

Parameter can be obligatory and optional. The optional parameter (0 or more) have to follow the obligatory ones.

Place to Define Functions

Functions have to be defined before they can be used:

```
f()

def f():
    print "This doesn't work!"


$ python function_def_wrong.py
Traceback (most recent call last):
  File "function_def_wrong.py", line 3, in
<module>
    f()
NameError: name 'f' is not defined
```


Functions: Example

```
#!/usr/bin/python

def add(x, y):
    """Returns x plus y."""
    return x + y

print add(2, 3)


>>> execfile("funktion1.py")
5
>>> add(4,5)
9
>>> add(8,3)
11
>>>
```


Functions: Example

```
#!/usr/bin/python

def add(x, y=5):
    """Returns x plus y."""
    return x + y

print add(2, 3)


>>> execfile("funktion1.py")
5
>>> add(4)
9
>>> add(8,3)
11
>>>
```


Docstring

The first statement in the body of a function is usually a string, which can be accessed by ***monty_python.__doc__***, if monty_python ist the name of the function.

This string is called **Docstring**.

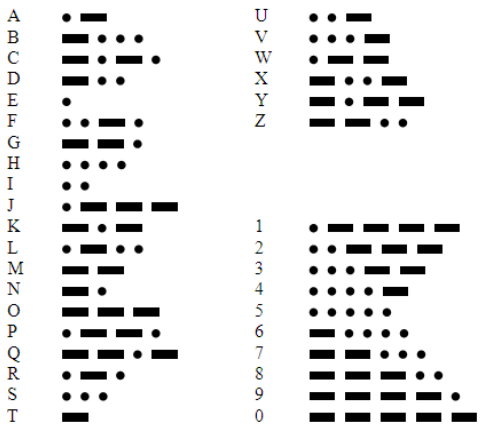
```
>>> execfile("funktion1.py")
>>> add.__doc__
'Returns x plus y.'
>>> add2.__doc__
'Returns x plus y.'
>>>
```

The Letter „E“

Exercise:
Write a function which counts the numbers of „e“ in a string.

```
def number_of_es(str):
   counter = 0
   for letter in str:
      if letter == "e":
         counter += 1;
   return counter

str =" "
while str:
   str = raw_input("Bitte String eingeben: ")
   print  number_of_es(str.lower())
```


Exercise: Morse Code



Write a function, which translates a text in morse code, i.e. the function returns a string with the morse code.

Exercise

Write a function with a 2-dimensional field and a row and column positon as parameters. The field (board) is filled with 0's and 1's. The function shall return the number of neighbours of the position.

0	0	0	0	0	0	0	0
0	1	1	0	0	0	1	0
0	0	0	0	0	1	1	0
0	1	1	0	1	0	1	0
0	1	1	1	0	0	0	0
0	1	0	0	1	0	1	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0

Number of neighbours of the green field: 4

Solution

```
def number_of_neighbours(board, row, col):
    """ counts the numbers of an array_element with
    the position row and column. It is assumed,
    that i,j > 0 and i,< < len(board)-1"""
    counter = 0
    for i in [row-1,row,row+1]:
        for j in [col-1,col,col+1]:
            counter += board[i][j]
    counter -= board[row][col]
    return counter

board = [ [0,0,0,0,0,0,0,0,0,0,0,0],
          [0,1,0,1,0,0,0,0,1,0,1,0],
          [0,1,0,1,0,1,0,1,0,0,0,0],
          [0,1,1,0,0,0,0,0,1,1,1,0],
          [0,0,0,1,1,0,0,0,0,0,0,0],
          [0,1,0,1,0,0,0,0,1,0,1,0],
          [0,0,0,0,0,0,0,0,0,0,0,0]]
while True:
   r = int(raw_input("row: "))
   if r == 0:
       break
   c = int(raw_input("col: "))
   print  "neighbours: ", number_of_neighbours(board, r,c)
```

Example: Heron Method

The "Babylonian method" or "Heron's method" is a quadratically convergent algorithm means that the number of correct digits of the approximation where the number of correct digits of the approximaiton roughly doubles with each iteration.

1) Begin with an arbitrary positive starting value x_0 (the closer to the root, the better).
2) Let x_{n+1} be the average of x_n and S / x_n (using the arithmetic mean to approximate the geometric mean).
3) Repeat step 2 until the desired accuracy is achieved.

$$x_{n+1} = \frac{x_n + \frac{a}{x_n}}{2}$$

Example: Heron Methos

```
def heron(a):
    """Approximates the square root of a"""
    eps = 0.0000001
    x = a / 2.0
    while True:
        y = (x + a/x) / 2.0
        if (x - y) < eps:
            break
        x = y
    return y
```


k^{th} Root

Exercise:

Write a function which calculates the k^{th} root of a number. The accuracy should be 0.00001.

$$x_{n+1} = \frac{(k-1)x_n^k + a}{kx_n^{k-1}}$$

Keyword Parameter

This is an alternative way to call a function.

The definition of the function doesn't change.

```
def sumsub(a, b, c=0, d=0):
    return a - b + c - d
```

```
>>> execfile("funktion1.py")
>>> sumsub(12,4)
8
>>> sumsub(12,4,27,23)
12
>>> sumsub(12,4,d=27)
```

Keyword parameters can only be those, which haven't been used as postional parameters.

Variable Length Argument List

Functions can use special arguments preceded with * characters to collect arbitrarily extra arguments (similar to the varargs feature in C).

```
def varlength(x, y, *more):
    print "x=", x, ", y=", y
    print "more: ", more
```

`x` and `y` are regular (positional) parameter, while `*more` references a tuple.

```
>>> varlength(3,4)
x= 3 , y= 4
more:  ()
>>> varlength(3,4,"Hello World", 42)
x= 3 , y= 4
more:  ('Hello World', 42)
```


Variable Length Argument List

Exercise:
Write a function which calculates the arithmetic mean of a variable number of values.

$$\bar{x}_{\mathbf{arithm}} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

Arbitrary Keyword Parameters

There is also a mechanism for an arbitrary number of keyword parameters.
To do this, we use the ** notation:

```
>>> def f(**args):
...     print(args)
...
>>> f()
{}
>>> f(de="German",en="English",fr="French")
{'fr': 'French', 'de': 'German', 'en': 'English'}
>>>
```


* in Function Calls

A * can appear in function calls as well:
The semantics is in this case “inverse” to a star in a function definition. An argument will be unpacked and not packed:

```
>>> def f(x,y,z):
...     print(x,y,z)
...
>>> p = (47,11,12)
>>> f(*p)
(47, 11, 12)
```

This call is definitely more comfortable than the following one:

```
>>> f(p[0],p[1],p[2])
(47, 11, 12)
>>>
```

** in Function Calls

The following example demonstrates the usage of ** in a function call:

```
>>> def f(a,b,x,y):
...     print(a,b,x,y)
…
>>> d = {'a':'append',
'b':'block','x':'extract','y':'yes'}
>>> f(**d)
('append', 'block', 'extract', 'yes')
```

And now in combination with *:

```
>>> t = (47,11)
>>> d = {'x':'extract','y':'yes'}
>>> f(*t, **d)
(47, 11, 'extract', 'yes')
>>>
```


Simple Definition of Dictionaries

The definition of Dictionaries with Strings as keywords is “stuffed” with quotes:

```
>>> colours = {'red':1, 'green':2, 'blue':3}
>>> def makedict(**dictargs):
...     return dictargs
...
```

This can be easierly achieved by using a function with “**”-argument:

```
>>> colours = makedict(red=1, green=2, blued=3)
>>> colours
{'green': 2, 'blued': 3, 'red': 1}
>>>
```


Parameter Passing in Detail

There are generally two kinds of parameter passing:

call-by-value

The argument expression is evaluated, and the resulting value is bound to the corresponding variable in the function (frequently by copying the value into a new memory region).

call-by-reference

The argument is a reference rather than a copy of a value. This typically means that the function can modify the argument. Call-by-reference has the advantage of greater time- and space-efficiency

call-by-value - call-by-reference

Any changes to a reference parameter inside of a function affect the main program as well. If this was not wanted, we call this a side effect.

There are no side effects with call-by-value, but the catch is the “copy”, i.e. especially if large lists and dictionaries are passed to a function.

Call by Object

Call-by-Object is also known as Call-by-Sharing or Call-by-Object-Sharing.

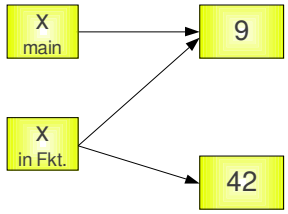
The semantics of call-by-Object differ from call-by-reference in that assignments to function arguments within the function aren't visible to the caller.
However since the function has access to the same object as the caller (no copy is made), mutations to those objects within the function are visible to the caller, which differs from call-by-value semantics.

Although this term has widespread usage in the Python community, identical semantics in other languages such as Java and Visual Basic are often described as call-by-value, where the value is implied to be a reference to the object.

Example

```
def ref_demo(x):
    print "x=",x," id=",id(x)
    x=42
    print "x=",x," id=",id(x)
```

```
>>> x=9
>>> ref_demo(x)
x= 9  id= 29488104
x= 42  id= 29489304
>>> id(x)
29488104
>>>
```



Side Effects

```
>>> def s(list):
...     print id(list)
...     list += [47,11]
...     print id(list)
...
>>> fib = [0,1,1,2,3,5,8]
>>> id(fib)
24746248
>>> s(fib)
24746248
24746248
>>> id(fib)
24746248
>>> fib
[0, 1, 1, 2, 3, 5, 8, 47, 11]
```


Side Effects

No side effect, because now it is not in-place anymore:

```
>>> def s(liste):
...     liste = liste + [47,11]
...
>>> fib = [0,1,1,2,3,5,8]
>>> s(fib)
>>> fib
[0, 1, 1, 2, 3, 5, 8]
```

Passing the Copy of a List

```
>>> def s(list):
...     list += [47,11]
...     print list
...
>>> fib = [0,1,1,2,3,5,8]
>>> s(fib[:])
[0, 1, 1, 2, 3, 5, 8, 47, 11]
>>> fib
[0, 1, 1, 2, 3, 5, 8]
```

The Slice Operator creates a copy of the list, i.e. `fib[:]` copies the complete list. This copy is passed on to the function with `s(fib)`.

Another Side Effect

```
>>> def f(list=[0,1]):
...     list += [list[-1]+list[-2]]
...     return list
...
>>> f()
[0, 1, 1]
>>> f()
[0, 1, 1, 2]
>>> f()
[0, 1, 1, 2, 3]
```

f is called without an argument for the optional parameter. The first time the optional parameter is used, but not the second time!

Explanation: The default values get assigned to the function when the function is defined, not when the function is called.

Without +=

```
>>> def f(list=[0,1]):
...     list = list + [list[-1]+list[-2]]
...     return list
...
>>> f()
[0, 1, 1]
>>> f()
[0, 1, 1]
>>> f()
[0, 1, 1]
```


Side Effects

```
def fos(list=None):
    if list is None:
        list=[0,1]
    list += [list[-1]+list[-2]]
    return list
```

```
>>> execfile("seiteneffekte.py")
>>> fib=fos()
>>> fib
[0, 1, 1]
>>> fos(fib)
[0, 1, 1, 2]
>>> fib
[0, 1, 1, 2]
>>> fos()
[0, 1, 1]
>>> fib
[0, 1, 1, 2]
```

Nested Functions

```
def hypotenuse(cathetus1, cathetus2):
    def square(x):
        return x * x

    def sum(a, b):
        return a + b

    hyp = sum(square(cathetus1),
square(cathetus2))
    return hyp

print "The square of the hypothenuse:",
hypotenuse(3, 4)
```

Namespaces and Scopes

A namespace is a mapping from names to objects.
Namespaces are usually implemented in Python as dictionaries.
A scope is a textual region of a Python program where a namespace is directly accessible.

Scopes in Python:

- the innermost scope contains the local names
- the scope of an enclosing function
- the next-to-last scope contains the current module's global names
- the outermost scope is the namespace containing built-in names

Global or Local

```
def f():
    print s
s = "Python"
f()
```

Output:
`Python`

```
def f():
    s = "Perl"
    print s

s = "Python"
f()
print s
```

Output:
`Perl`
`Python`

Global and Local

```
def f():
   print s
   s = "Perl"
   print s


s = "Python"
f()
print s
```

If we execute the previous script, we get the error message:

```
UnboundLocalError:
local variable 's'
referenced before
assignment
```

The variable `s` is ambigious in `f()`, i.e. in the first `print` in `f()` the global `s` could be used with the value `"Python"`. After this we define a local variable `s` with the assignment `s = "Perl"`

Global Variables in Functions

So far, we are capable of reading global variables, but we can't change them.

If you need read and write access, the variable has to be denoted as global with the keyword "`global`"

```
def f():
    global s
    print s
    s = "dog"
    print s


s = "cat"
f()
print s
```

output:

```
cat
dog
dog
```


Global Variables of Functions

Local variables of functions can't be accessed, when the function call has finished.

```
def f():
    s = "cat"
    print s

f()
print s
```

Output of the script:

```
cat
Traceback (most recent call last):
  File "global_local3.py", line 6,
in <module>
    print s
NameError: name 's' is not defined
```


Another Example

```
def foo(x, y):
    global a
    a = 42
    x,y = y,x
    b = 33
    c = 100
    print a,b,x,y

a,b,x,y = 1,15,3,4
foo(17,4)
print a,b,x,y
```

```
42 33 4 17
```

```
42 15 3 4
```

map(),filter() and reduce()

The lambda Operator

lambda allows you to create an unnamed throw-away function.

```
lambda argument_list: expression
```

After the keyword lambda follows an unnamed function.
Before the colon (:) you list the parameters of the function.
After the colon you formulate what the function shall return.
You can assign the function to a variable to give it a name. The following two practices are identical:

```
>>> def add42(x):
...     return x + 42
...
>>> add42(10)
52
>>> add42 = lambda x : x + 42
>>> add42(10)
52
```


The map Function

The advantage of the lambda Operator can be seen in combination with the map-Function.

```
r = map(func, seq)
```

`map` applies the function `func` on all the elements of the sequence `seq`.

```
def fahrenheit(T):
    return ((float(9)/5)*T + 32)
def celsius(T):
    return (float(5)/9)*(T-32)
temp = (36.5, 37, 37.5,39)

F = map(fahrenheit, temp)
C = map(celsius, F)
```


Using lambda Notation

```
temp = (36.5, 37, 37.5,39)

K = 5.0 / 9

F = map(lambda T: 1.8 * T + 32, temp)
C = map(lambda T: K * (T-32), F)
```

map() on Multiple Lists

The map() Function can work simultaneously on more than one list.

These lists have to have the same length.

```
>>> a = [1,2,3,4]
>>> b = [17,12,11,10]
>>> c = [-1,-4,5,9]
>>> map(lambda x,y:x+y, a,b)
[18, 14, 14, 14]
>>> map(lambda x,y,z:x+y+z, a,b,c)
[17, 10, 19, 23]
>>> map(lambda x,y,z:x+y-z, a,b,c)
[19, 18, 9, 5]
```


Filtering of a List

The function **`filter(function, list)`** provides an elegant way to filter out those elements which are True if applied to the function.

```
fib = [0,1,1,2,3,5,8,13,21,34]

def is_even(x):
  return x % 2 == 0

result = filter(is_even, fib)
print result
```

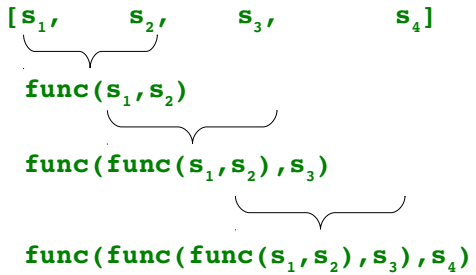
Exercise: Write a call to this function using a lambda Operator

Solution

```
fib = [0,1,1,2,3,5,8,13,21,34]

result = filter(lambda x: x%2==0,fib)
print result
```


`reduce` Function

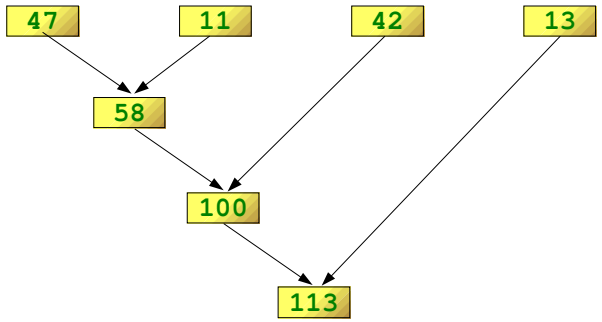
The function `reduce(func, seq)` continually applies func() on the sequence seq. The arguments of func are replaced by the result of func() in the list.



Example for reduce()

```
>>> reduce(lambda x,y: x+y, [47,11,42,13])
113
```

Illustration of the example:



Exercises for reduce()

Calculate the sum of the numbers from 1 to 100 by using reduce().

```
>>> reduce(lambda x, y: x+y, range(1,101))
5050
```

Calculate with reduce() the product of the numbers 1 to 9.

```
>>> reduce(lambda x, y: x*y, range(1,10))
362880
```


Exercise

How can you determine the maximum of a list of numerical values by using reduce()?

```
>>> f = lambda a,b: a if (a > b) else b
>>> reduce(f, [47,11,42,102,13])
102
>>>
```


Criticism by Guido van Rossum

About 12 years ago, Python aquired lambda, reduce(), filter() and map(), courtesy of (I believe) a Lisp hacker who missed them and submitted working patches. But, despite of the PR value, I think these features should be cut from Python 3000.

*„So now reduce(). This is actually the one I've always hated most, because, apart from a few examples involving + or *, almost every time I see a reduce() call with a non-trivial function argument, I need to grab pen and paper to diagram what's actually being fed into that function before I understand what the reduce() is supposed to do. So in my mind, the applicability of reduce() is pretty much limited to associative operators, and in all other cases it's better to write out the accumulation loop explicitly."*

Quelle:
http://www.artima.com/weblogs/viewpost.jsp?thread=98196

List Comprehension

List comprehension can be used instead of `map()` and `filter()`:

```
>>> list = range(1,10)
>>> [x+3 for x in list]
[4, 5, 6, 7, 8, 9, 10, 11, 12]
```

The `for-in`-branch can be extended by a case differentiation:

```
>>> fib = [0,1,1,2,3,5,8,13,21,34]
>>> [x for x in fib if x%2 == 0]
[0, 2, 8, 34]
```


List Comprehension

The following notation are used in mathematics to describe sets:

$S = \{x^2 : x \text{ in } \{0 \ldots 9\}\}$

$V = (1, 2, 4, 8, \ldots, 2^{12})$

$M = \{x \mid x \text{ in } S \text{ and } x \text{ even}\}$

In Python it can be done like this:

```
>>> S = [x**2 for x in range(10)]
>>> V = [2**i for i in range(13)]
>>> M = [x for x in S if x % 2 == 0]
>>>
```


List Comprehension

The Python lists look like this:

```
>>> print S; print V; print M
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512,
1024, 2048, 4096]
[0, 4, 16, 36, 64]
>>>
```


List-Comprehension II

More complicated terms can be calculated within list comprehension as well:

```
>>> x = [2, 4, 6, 8]
>>> y = [3, 0, -5, 1]
>>> [x[i] + y[i] for i in range(4)]
[5, 4, 1, 9]
```

A list comprehension can consist of an arbitrary number of for/in's. These can be viewed as nested for loops:

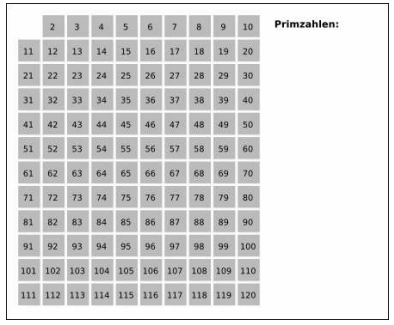
```
>>> l1 = ["house","car"]
>>> l2 = ["red","green","blue"]
>>> [(x,y) for x in l1 for y in l2]
[('house', 'red'), ('house', 'green'), ('house',
'blue'), ('car', 'red'), ('car', 'green'),
('car', 'blue')]
```

A more Challenging Example

Calculation of the prime numbers up to 100:

```
>>> noprimes = [j for i in range(2, 8) for j in
range(i*2, 100, i)]
>>> primes = [x for x in range(2, 100) if x not in
noprimes]
>>> print primes
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
>>>
```


Visualisation



From Wikipedia

Problem of the Previous Example

```
>>> from math import sqrt
>>> n = 100
>>> sqrt_n = int(sqrt(n))
>>> no_p = [j for i in range(2,sqrt_n) for j in range(i*2,
n, i)]
>>> no_p
[4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32,
34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62,
64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92,
94, 96, 98, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39,
42, 45, 48, 51, 54, 57, 60, 63, 66, 69, 72, 75, 78, 81, 84,
87, 90, 93, 96, 99, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44,
48, 52, 56, 60, 64, 68, 72, 76, 80, 84, 88, 92, 96, 10, 15,
20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90,
95, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72, 78, 84, 90,
96, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98, 16,
24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 18, 27, 36, 45, 54,
63, 72, 81, 90, 99]
>>>
```

Problem: There are lots of double entries in this list!

Optimization

Solution: We have to check, if an element is already in the list while the list is being built.

This is possible, because the Python interpreter creates a “secret” variable, which exists only while the list is being built and it contains a reference to this list:

```
locals()['_[1]']
>>> no_p = [j for i in range(2,sqrt_n) for j in range(i*2,
n, i) if j not in locals()['_[1]']]
>>> no_p
[4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32,
34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60,
62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88,
90, 92, 94, 96, 98, 9, 15, 21, 27, 33, 39, 45, 51, 57, 63,
69, 75, 81, 87, 93, 99, 25, 35, 55, 65, 85, 95, 49, 77,
91]
```

Another Example

```
>>> words = 'The quick brown fox jumps over the lazy
dog'.split()
>>> stuff = [[w.upper(), w.lower(), len(w)] for w in
words]
>>> for i in stuff:
...     print i
...
['THE', 'the', 3]
['QUICK', 'quick', 5]
['BROWN', 'brown', 5]
['FOX', 'fox', 3]
['JUMPS', 'jumps', 5]
['OVER', 'over', 4]
['THE', 'the', 3]
['LAZY', 'lazy', 4]
['DOG', 'dog', 3]
>>>
```


Exercises

Write a list comprehension, which generates all possible combinations of two dice.

```
comb = [(i,j) for i in range(1,7) for j in range(1,7)]
```

Write a list comprehension which inverses a given dictionary, i.e. the values will be keys and the keys will be turned into values.

```
>>> dir = {'DE':'Deutschland','FR':'Frankreich'}
>>> inv = dict((dir[i],i) for i in dir)
>>> print inv
{'Deutschland': 'DE', 'Frankreich': 'FR'}
>>>
```


Set Comprehension in Python 3

Set Comprehension uses curly braces like dictionaries:

```
>>> s = { x for x in "set comprehension"}
>>> print(s)
{' ', 'c', 'e', 'i', 'h', 'm', 'o', 'n', 'p',
's', 'r', 't'}
>>> type(s)
<class 'set'>
```

What about empty sets?

```
>>> s = {}
>>> type(s)
<class 'dict'>
>>>
```

A list comprehension in comparison:

```
>>> l = [ x for x in "set comprehension"]
>>> type(l)
<class 'list'>
>>>
```


Exercise

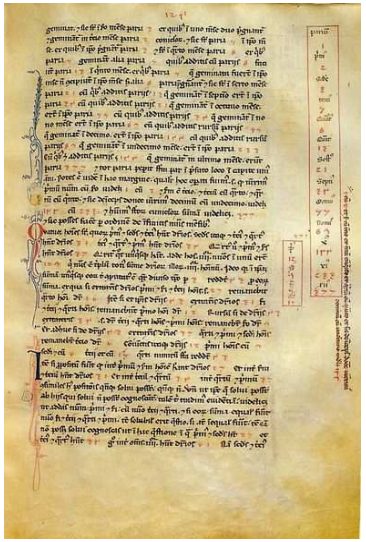
Write an optimised version for calculating the prime numbers using Python3 and set abstractions.

Fibonacci

Leonardo da Pisa, better known as Fibonacci, meaning “figlio de Bonacio” (1180 - 1241)

He was one of the most important mathematicians of the Middle Ages

His major work:
Liber abbaci

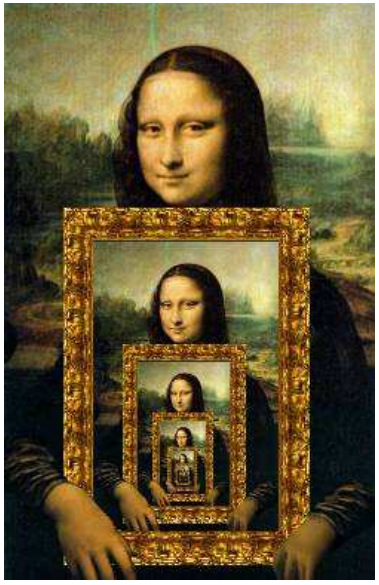


Recursive Functions

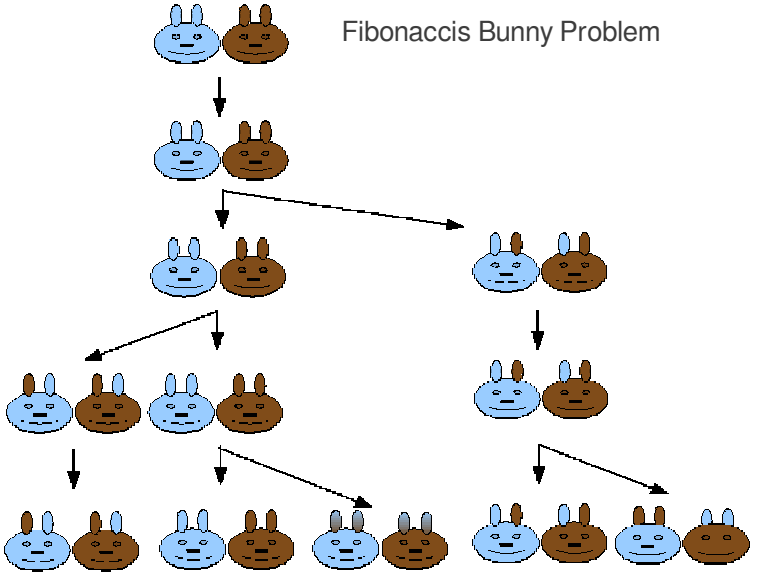
Recursion in computer science is a method where the solution to a problem depends on solutions to smaller instances of the same problem.

A recursive function is a function calling itself.

latin origin:
recurrere, to run back;



Fibonaccis Bunny Problem



Exercise

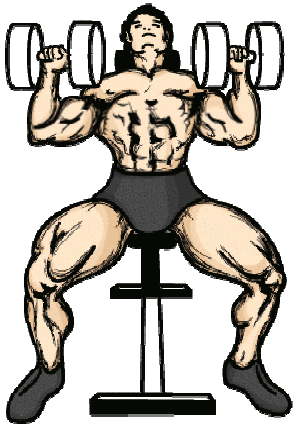
Definition: Every item of the Fibonacci sequence is calculated by its predecessors.

The first eight are:

0, 1, 1, 2, 3, 5, 8, und 13.

Formal Definition:
F(n) = F(n-1) + F(n-2),
where F(0)=0 and F(1)=1

Exercise:
Write a program to calculate F(n)



Recursive Solution

```
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

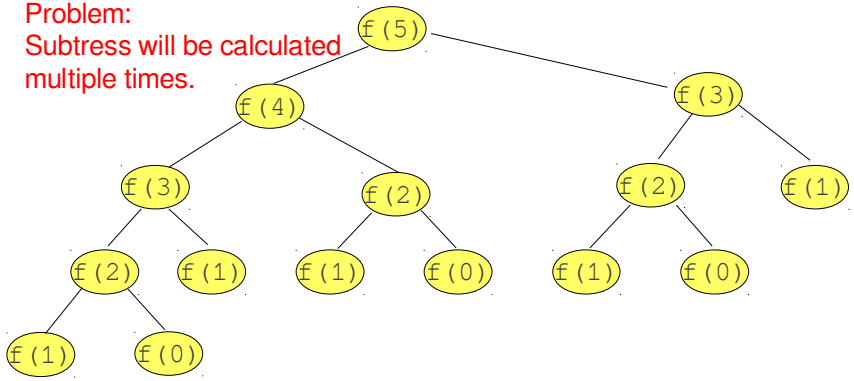
Calling the function:

```
>>> execfile("fibonacci.py")
>>> fib(5)
5
>>> fib(0), fib(1), fib(2), fib(3)
(0, 1, 1, 2)
```


Problem

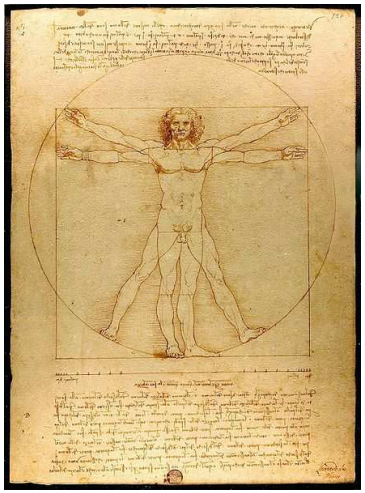
If you call the previous function e.g. for f(5) (we use f instead of fib), you get the following execution tree:

Problem:
Subtress will be calculated multiple times.



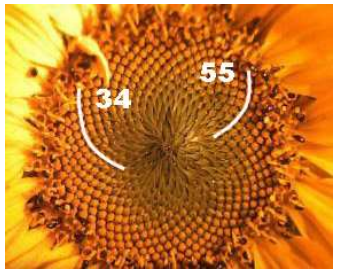
Iterative Solution

```
def fib(n):
    a, b = 0, 1
    for i in range(n-1):
        a, b = b, a + b
    return b
```

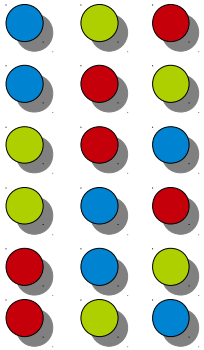


Recursion with Memoization

```
memo = {0:0, 1:1}
def fib(n):
    if not n in memo:
        memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```



Permutations



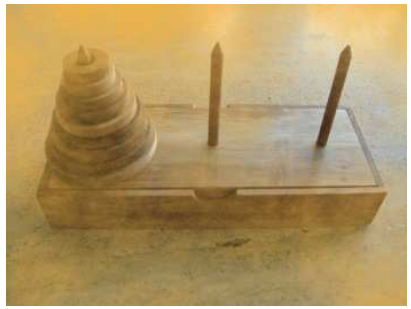
Every possible arrangement of n elements, in which all n elements are used, is called a **Permutation** of these elements.

The number of permutations are defined as:

```
n! = n*(n-1)*(n-2) ... 2 * 1
```

Exercise:
Write a function which calculates the **faculty** (`n!`) of a number n.

The Towers of Hanoi



According to an old legend there is a monastery with three poles.
One of them filled with 64 gold disks.
The disks are of different sizes, and they are put on top of each other, according to their size, i.e. each disk on the pole a little smaller than the one beneath it.
The monks have to move this stack from one of the three poles to another one. But one rule has to be applied: a large disk can never be placed on top of a smaller one.

When they would have finished their work, the legend tells, the temple would crumble into dust, and the world would end.

But don't be afraid, it's not very likely, that they will finish their work soon, because 2^{64-1} moves are necessary, i.e. 18,446,744,073,709,551,615 to move the tower according to the rules.

Rules of the Game

The game uses three rods. A number of disks is stacked in decreasing order from the bottom to the top of one rod, i.e. the largest disk at the bottom and the smallest one on top. The disks build a conical tower.

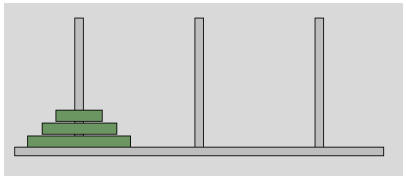
The aim is to move the tower of disks from one rod to another rod.

The following rules have to be obeyed:

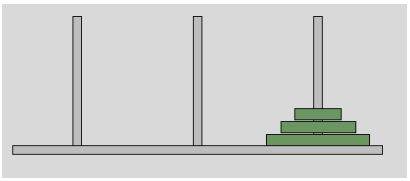
- Only one disk may be moved at a time.
- Only the most upper disk from one of the rods can be moved in a move
- It can be put on another rod, if this rod is empty or if the most upper disk of this rod is larger than the one which is moved.

Playing Around

Try to find a way to turn this configuration



into this configuration by applying the rules of the game



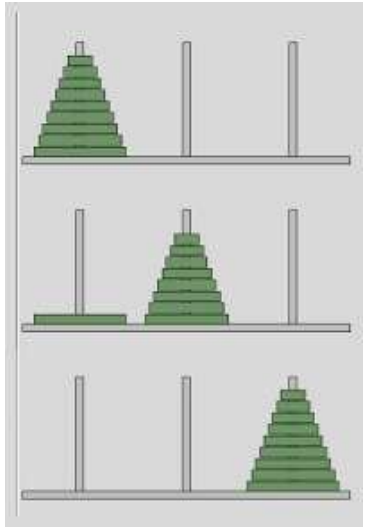
The Recursion Step

There is a general rule for moving a tower of size n (n > 1) from the peg S to the peg T:

move a tower of n - 1 discs D_{n-1} ... D_1 from S to A. Disk D_n is left alone on peg S

Move disk D_n to T

move the tower of n - 1 discs D_{n-1} ... D1 on A to T, i.e. this tower will be put on top of Disk D_n



The Python Implementation

```
def hanoi(n, source, helper, target):
    if n > 0:
        # move tower of size n - 1 to helper:
        hanoi(n - 1, source, target, helper)
        # move disk from source peg to target peg
        if source:
            target.append(source.pop())
        # move tower of size n-1 from helper to
target
        hanoi(n - 1, helper, source, target)

source = [4,3,2,1]
target = []
helper = []
hanoi(len(source),source,helper,target)
print source, helper, target
```


How does it Work?

```
def hanoi(n, source, helper, target):
    print "hanoi( ", n, source, helper, target, "
called"
    if n > 0:
        hanoi(n - 1, source, target, helper)
        if source[0]:
            disk = source[0].pop()
            print "moving " + str(disk) + " from
" + source[1] + " to " + target[1]
            target[0].append(disk)
        hanoi(n - 1, helper, source, target)

source = ([4,3,2,1], "source")
target = ([], "target")
helper = ([], "helper")
hanoi(len(source[0]),source,helper,target)
print source, helper, target
```

Sieve of Eratosthenes

Implementation of the Sieve of Eratosthenes as a recursive function:

```
from math import sqrt
def primes(n):
   if n == 0:
      return []
   elif n == 1:
      return [1]
   else:
      p = primes(int(sqrt(n)))
      no_p = [j for i in p for j in range(i*2, n,
i) if j not in locals()['_[1]']]
      p = [x for x in range(2, n) if x not in
no_p]
   return p
```


Exercise

$$\frac{1}{1} = 1 \qquad \frac{2}{1} = 1+\frac{1}{1} \qquad \frac{3}{2} = 1+\frac{1}{1+\frac{1}{1}} \qquad \frac{5}{3} = 1+\frac{1}{1+\frac{1}{1+\frac{1}{1}}} \qquad \frac{8}{5} = 1+\frac{1}{1+\frac{1}{1+\frac{1}{1+\frac{1}{1}}}}$$

Write a recursive function to implement this continued fraction (approximation of the golden ratio)

corresponds to:

$$\frac{f_{n+1}}{f_n} \approx a = \left(\frac{1+\sqrt{5}}{2}\right) = \Phi \approx 1{,}618...$$

Exercise

The greatest common divisor (gcd) (also called “greatest common denominator”) of two numbers (integers) a and b satisfies the following condition:
If a == b, then gcd(a,b) == a
If a > b, then gcd(a,b) equals gcd((a - b) ,b)
If a < b, then gcd(a,b) equals gcd(a ,(b - a))

Write a recursive function, which calculates the gcd of two numbers.